

오픈소스프로젝트 제품 구성,배포 및 운영 자료

본 문서는 Caffit 프로젝트의 실제 배포 및 운영 경험을 가정하고 작성되었습니다. AWS EC2 환경에서 Docker 컨테이너를 활용하여 백엔드와 AI 모듈을 배포하고 운영했다고 가정한 후에 내용을 정리하였습니다.

1. 시스템 구성 개요

1.1 기술 스택 구성표

계층	기술 스택	용도	실행 환경
클라이언트	React Native + TypeScript	모바일 애플리케이션	Android/iOS 디바이스
API 서버	Spring Boot (Java 17)	RESTful API 제공	Docker Container (Port: 8080)
AI 서비스	Flask (Python 3.9)	LLM 기반 음료 인식 및 추천	Docker Container (Port: 5001)
데이터 저장소	MySQL 8.0	사용자 데이터 및 섭취 기록	AWS RDS 또는 EC2 내부
배포 플랫폼	AWS EC2	서버 인프라	Amazon Linux 2023
컨테이너화	Docker	애플리케이션 패키징 및 배포	-
CI/CD	GitHub Actions	자동 빌드 및 배포	-

1.2 시스템 아키텍처

Caffit 프로젝트는 **3-tier 아키텍처**로 설계되었으며, 각 계층은 독립적으로 운영됩니다.

클라이언트 계층

React Native로 개발된 모바일 애플리케이션이 사용자 인터페이스를 제공합니다. Android와 iOS 플랫폼을 모두 지원하며, 네이티브 기능(카메라, GPS 등)을 활용합니다.

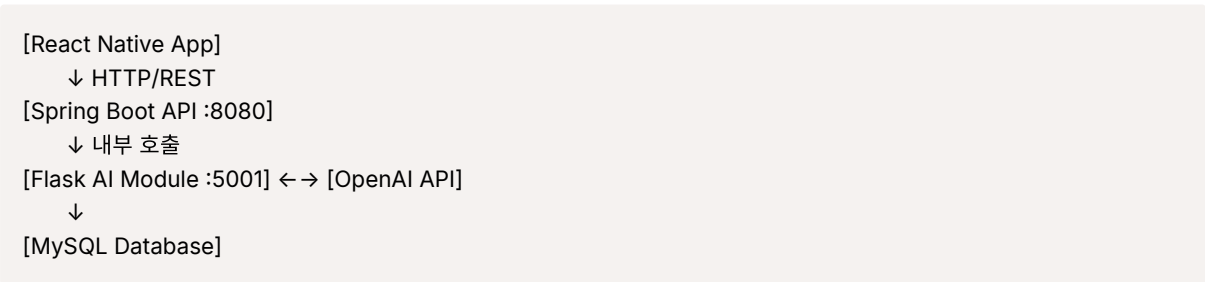
서비스 계층

Spring Boot 기반의 REST API 서버가 비즈니스 로직을 처리하고, Flask 기반의 AI 모듈이 LLM과 RAG 기술을 활용하여 음료 인식 및 맞춤형 추천 서비스를 제공합니다.

데이터 계층

MySQL 데이터베이스에 사용자 정보, 카페인 섭취 기록, 수면 데이터 등이 저장됩니다.

1.3 서비스 간 통신 구조



2. 배포 전략 및 절차

2.1 배포 환경 사전 준비

필수 요구사항

- AWS 계정 및 EC2 인스턴스 (t2.micro 이상 권장)
- Docker 설치 (로컬 및 EC2)
- MySQL 데이터베이스 (AWS RDS 또는 EC2 내부 설치)
- 필요한 API 키 (Kakao REST API, OpenAI API)

네트워크 설정

EC2 보안 그룹에서 다음 포트를 개방해야 합니다:

- **22번 포트:** SSH 접속용
- **8080번 포트:** Spring Boot API 서버
- **5001번 포트:** Flask AI 모듈
- **3306번 포트:** MySQL (내부 통신용, 선택사항)

2.2 백엔드 서버 배포

Step 1: 로컬 빌드 및 이미지 생성

로컬 개발 환경에서 다음 명령어를 순서대로 실행합니다:

```
# 1. 프로젝트 디렉토리로 이동cd backend

# 2. Gradle을 사용한 애플리케이션 빌드 (테스트 제외)
./gradlew build -x test

# 3. Docker 이미지 빌드 (Linux AMD64 아키텍처)
docker build --platform=linux/amd64 -t caffit-backend:latest .

# 4. 이미지를 tar 파일로 저장
docker save caffit-backend:latest > caffit-backend.tar

# 5. EC2 인스턴스로 이미지 파일 전송
scp -i <pem-키-경로> caffit-backend.tar ec2-user@<EC2-IP-주소>:~/
```

Step 2: EC2에서 컨테이너 실행

EC2 인스턴스에 SSH 접속 후 다음 작업을 수행합니다:

```
# 1. 전송된 이미지 파일 로드
docker load < caffit-backend.tar

# 2. 기존 컨테이너가 있다면 중지 및 삭제
docker stop caffit-backend 2>/dev/null || true
docker rm caffit-backend 2>/dev/null || true

# 3. 새 컨테이너 생성 및 실행
docker run -d \
  --name caffit-backend \
  -p 8080:8080 \
  -e DB_URL=jdbc:mysql://<DB_HOST>:3306/caffit?useSSL=false&serverTimezone=Asia/Seoul \
  -e DB_USER=<데이터베이스_사용자명> \
  -e DB_PASSWORD=<데이터베이스_비밀번호> \
  -e KAKAO_REST_API_KEY=<카카오_API_키> \
  --restart unless-stopped \
  caffit-backend:latest

# 4. 컨테이너 실행 상태 확인
docker ps | grep caffit-backend
docker logs caffit-backend
```

2.3 AI 모듈 배포

Step 1: 로컬 빌드 및 이미지 생성

```
# 1. AI 모듈 디렉토리로 이동cd AI_model

# 2. Docker 이미지 빌드
docker build --platform=linux/amd64 -t caffit-ai:latest .

# 3. 이미지를 tar 파일로 저장
docker save caffit-ai:latest > caffit-ai.tar

# 4. EC2로 전송
scp -i <pem-키-경로> caffit-ai.tar ec2-user@<EC2-IP-주소>:~/
```

Step 2: EC2에서 컨테이너 실행

```
# 1. 이미지 로드
docker load < caffit-ai.tar

# 2. 기존 컨테이너 정리
docker stop caffit-ai 2>/dev/null || true
docker rm caffit-ai 2>/dev/null || true

# 3. 컨테이너 실행
docker run -d \
  --name caffit-ai \
  -p 5001:5001 \
  -e OPENAI_API_KEY=<OpenAI_API_키> \
  -e DB_URL=mysql://<DB_HOST>:3306/caffit \
  --restart unless-stopped \
  caffit-ai:latest

# 4. 실행 확인
docker ps | grep caffit-ai
docker logs caffit-ai
```

2.4 자동 배포 파이프라인 구성

GitHub Actions를 활용한 자동 배포를 설정하려면 [.github/workflows/](#) 디렉토리에 워크플로우 파일을 구성합니다. 코드 변경사항이 main 브랜치에 푸시되면 자동으로 빌드 및 배포가 진행됩니다.

참고: 자세한 설정은 프로젝트 루트의 [.github/workflows/deploy-backend.yml](#) 및 [deploy-ai.yml](#) 파일을 참조하세요.

2.5 배포 검증

배포가 완료된 후 다음 방법으로 서비스 상태를 확인합니다:

```
# Health Check 엔드포인트 호출
curl http://<EC2-IP-주소>:8080/api/health

# 예상 응답: {"status":"UP"}
```

브라우저에서 <http://<EC2-IP-주소>:8080/api/health> 에 접속하여 JSON 응답을 확인할 수도 있습니다.

3. 운영 관리 가이드

3.1 일상 운영 체크리스트

서비스 시작 절차

EC2 인스턴스가 중지된 상태에서 시작할 때는 다음 순서로 진행합니다:

- ☐ EC2 인스턴스 시작 확인 (AWS 콘솔)
- ☐ SSH 접속 가능 여부 확인
- ☐ MySQL 서비스 실행 상태 확인 (`sudo systemctl status mysqld`)
- ☐ 백엔드 컨테이너 시작 (`docker start caffit-backend`)
- ☐ AI 모듈 컨테이너 시작 (`docker start caffit-ai`)
- ☐ 각 서비스 Health Check 수행

서비스 중지 절차

과금 방지를 위해 서비스를 중지할 때:

- ☐ 백엔드 컨테이너 중지 (`docker stop caffit-backend`)
- ☐ AI 모듈 컨테이너 중지 (`docker stop caffit-ai`)
- ☐ (선택사항) EC2 인스턴스 중지

3.2 모니터링 및 로그 관리

컨테이너 상태 모니터링

```
# 실행 중인 모든 컨테이너 확인
docker ps

# 특정 컨테이너 상세 정보
docker inspect caffit-backend

# 리소스 사용량 실시간 모니터링
docker stats caffit-backend caffit-ai
```

로그 확인 방법

```
# 최근 로그 확인 (기본 100줄)
docker logs --tail 100 caffit-backend

# 실시간 로그 스트리밍
docker logs -f caffit-backend

# 특정 시간 이후 로그만 확인
docker logs --since 2025-01-15T10:00:00 caffit-backend

# 로그를 파일로 저장
docker logs caffit-backend > backend_$(date +%Y%m%d).log
```

시스템 리소스 모니터링

EC2 인스턴스의 리소스 사용량은 AWS 콘솔의 CloudWatch를 통해 확인할 수 있습니다. CPU 사용률, 메모리 사용량, 네트워크 I/O 등을 모니터링하여 필요시 인스턴스 타입을 조정합니다.

3.3 데이터베이스 관리

백업 절차

```
# 전체 데이터베이스 백업
mysqldump -u caffit_user -p caffit > backup_$(date +%Y%m%d_%H%M%S).sql

# 특정 테이블만 백업
mysqldump -u caffit_user -p caffit intake sleep_log > tables_backup.sql
```

복원 절차

```
# 백업 파일로부터 복원
mysql -u caffit_user -p caffit < backup_20250115_120000.sql
```

3.4 업데이트 및 배포

수동 업데이트 절차

1. 로컬에서 코드 수정 및 테스트 완료
2. 새로운 Docker 이미지 빌드
3. 이미지를 EC2로 전송
4. 기존 컨테이너 중지 및 삭제
5. 새 컨테이너 실행
6. 서비스 정상 동작 확인

롤백 절차

문제 발생 시 이전 버전으로 되돌리기:

```
# 이전 이미지로 컨테이너 재생성
docker stop caffit-backend
docker rm caffit-backend
docker run -d --name caffit-backend -p 8080:8080 [환경변수] caffit-backend:previous-version
```

4. 인프라 구성 상세

4.1 Docker 컨테이너 구성

백엔드 컨테이너 설정

Dockerfile 구조:

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

환경 변수:

- **DB_URL**: MySQL 데이터베이스 연결 URL
- **DB_USER**: 데이터베이스 사용자명
- **DB_PASSWORD**: 데이터베이스 비밀번호
- **KAKAO_REST_API_KEY**: 카카오 맵 API 키

AI 모듈 컨테이너 설정

Dockerfile 구조:

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5001
CMD ["python", "-m", "AI_model.app"]
```

환경 변수:

- `OPENAI_API_KEY`: OpenAI API 인증 키
- `DB_URL`: 데이터베이스 연결 정보 (선택사항)

4.2 네트워크 구성

각 서비스는 독립적인 포트를 사용하여 통신합니다:

- **8080 포트**: Spring Boot API 서버 (외부 접근 가능)
- **5001 포트**: Flask AI 모듈 (외부 접근 가능)
- **3306 포트**: MySQL 데이터베이스 (내부 통신, 보안 그룹에서 제한 가능)

4.3 보안 고려사항

- 환경 변수에 민감한 정보(비밀번호, API 키) 저장
- EC2 보안 그룹에서 필요한 포트만 개방
- 데이터베이스 접근은 내부 네트워크로 제한 권장
- 정기적인 보안 업데이트 및 패치 적용

5. 문제 해결 가이드

5.1 일반적인 문제

컨테이너가 시작되지 않는 경우

- 로그 확인: `docker logs caffit-backend`
- 환경 변수 확인: `docker inspect caffit-backend | grep Env`
- 포트 충돌 확인: `netstat -tlnp | grep 8080`

데이터베이스 연결 실패

- MySQL 서비스 상태 확인: `sudo systemctl status mysqld`
- 연결 정보 확인 (환경 변수)
- 방화벽 규칙 확인

API 응답 없음

- 컨테이너 실행 상태 확인: `docker ps`
- 보안 그룹 규칙 확인 (AWS 콘솔)
- Health Check 엔드포인트 테스트

5.2 성능 최적화

- 불필요한 로그 레벨 조정
- 데이터베이스 쿼리 최적화
- 컨테이너 리소스 제한 설정 (필요시)